

SMS to SQLite

Prototype Implementation Plan

Water Safety Monitoring System — Backend v1

1. Overview

This document describes the implementation of the first working prototype of the SMS illness reporting pipeline. The goal is a minimal, working system where a person texts a station number to a Twilio phone number, the message is received by a Flask web server, the station ID is parsed and validated, the relevant sensor log entries are labelled as unsafe, and a confirmation SMS is sent back to the reporter.

This prototype is Generation 1 — its purpose is data collection and label generation, not live inference. The ML model will be trained separately once enough labelled data has accumulated.

2. System architecture

The pipeline has five components:

Component	Technology	Purpose
SMS receiver	Twilio	Receives texts, fires webhook to Flask
Web server	Flask (Python)	Parses message, runs business logic
Database	SQLite	Stores sensor logs and illness reports
Labelling logic	Python	Retrospectively labels sensor readings
Auto-reply	Twilio + Flask	Confirms receipt to the reporter

3. Prerequisites

Before writing any code, the following accounts and tools must be in place:

Accounts

- Twilio trial account — [twilio.com](https://www.twilio.com) (free, includes a real phone number and trial credit)
- Railway account — railway.app (free tier, for hosting the Flask app)
- GitHub account — to connect the repo to Railway

Local tools

- Python 3.10 or higher

- pip (Python package manager)
- Git
- ngrok — for exposing localhost to Twilio during development (ngrok.com, free tier)

4. File structure

The repository should contain the following files. All five are already written and available — do not rewrite them from scratch.

File	Purpose
app.py	Main Flask application — SMS webhook, routing, auto-reply
database.py	Database initialisation and connection helpers
labels.py	Retrospective labelling logic and CSV export
sensor_ingest.py	Blueprint for receiving sensor readings from the field device
requirements.txt	Python dependencies — Flask, Twilio, gunicorn
Procfile	Tells Railway how to run the app (gunicorn app:app)
.env.example	Template for environment variables — copy to .env locally

Note: Do not push .env to GitHub. It is listed in .gitignore already.

5. Environment variables

Two environment variables must be set before the app will run correctly:

Variable	Value	Where to set
DEVICE_SECRET	Any random string, e.g. wq-secret-2024	Railway dashboard + local .env
DATABASE_PATH	water_safety.db (default)	Optional — only change if needed

To set locally, copy .env.example to .env and fill in the values. On Railway, set them in the Variables tab of the project dashboard.

6. Implementation steps

Step 1 — Install dependencies

In the project directory, run:

```
pip install -r requirements.txt
```

Step 2 — Register the sensor blueprint

In `app.py`, add these two lines near the top, after the Flask app is created:

```
from sensor_ingest import sensor_bp
app.register_blueprint(sensor_bp)
```

This is the one manual step not already in the code. Without it, the `/ingest` endpoint will return 404.

Step 3 — Run locally

Start the Flask development server:

```
python app.py
```

The server will start on `http://localhost:5000`. The database will be created automatically on first run. Verify it worked by visiting:

```
http://localhost:5000/health
```

You should see: `{"status": "ok", "timestamp": "..."}"`

Step 4 — Expose to Twilio using ngrok

Twilio needs a public URL to send webhook requests to. In a new terminal window, run:

```
ngrok http 5000
```

ngrok will display a public URL, e.g. `https://abc123.ngrok.io`. Copy this — you will need it in Step 5. Note that this URL changes every time you restart ngrok on the free tier.

Step 5 — Configure Twilio webhook

- Log in to `console.twilio.com`
- Navigate to Phone Numbers > Manage > Active Numbers
- Click your Twilio number
- Under Messaging Configuration, set the webhook URL to:
`https://abc123.ngrok.io/sms`
- Set the HTTP method to POST
- Click Save

Step 6 — Test the SMS pipeline

From any phone, send a text to your Twilio number containing just a station number:

You should receive an automated reply within a few seconds: "Thank you. Station 4 has been flagged for review."

To verify the database was updated, visit:

`http://localhost:5000/reports`

You should see the illness report listed with a timestamp and confidence weight.

Step 7 — Test invalid inputs

Test the error handling by sending:

- A message with no number, e.g. "hello" — should receive a helpful reply asking for a station number
- A station number that does not exist, e.g. "99" — should receive a not found reply
- Multiple reports from the same station — confidence weight should increase from 0.3 to 0.6 to 0.9

Step 8 — Deploy to Railway

- Push the repository to GitHub
- Go to railway.app and create a new project
- Select Deploy from GitHub repo and connect your repository
- Railway will detect the Procfile automatically and deploy
- In the Variables tab, add `DEVICE_SECRET` with your chosen value
- Once deployed, Railway will show a public URL, e.g. `https://yourapp.railway.app`
- Update the Twilio webhook URL to use the Railway URL instead of the ngrok URL:
`https://yourapp.railway.app/sms`

Railway's free tier keeps the app always awake. No cold start delays unlike Render's free tier.

Step 9 — Verify production deployment

Send another test SMS to your Twilio number and confirm the reply arrives. Then check the production logs in the Railway dashboard to confirm no errors.

7. Testing checklist

Before handing over to the next development phase, confirm each of the following:

Test	Expected result	Pass?
GET /health	Returns {status: ok}	
GET /stations	Returns 4 seeded stations	

Test	Expected result	Pass?
Text '4' to Twilio number	Receives confirmation SMS	
GET /reports	Report visible with confidence 0.3	
Text '4' again within 7 days	Confidence increases to 0.6	
Text 'hello' (no number)	Receives helpful error reply	
Text '99' (invalid station)	Receives station not found reply	
POST to /ingest with valid JSON	Returns 201, reading saved	
GET /station/4/latest	Returns most recent reading	
Deploy to Railway	App live at railway URL	

8. Database schema reference

SQLite creates the database file automatically on first run. The schema has four tables:

stations

Column	Type	Purpose
id	INTEGER PRIMARY KEY	Station number — matches what people text
name	TEXT	Human readable name, e.g. Borehole A
location	TEXT	Description of physical location
install_date	TEXT	ISO date string

sensor_logs

Column	Type	Purpose
id	INTEGER	Auto-incrementing primary key
station_id	INTEGER	Foreign key to stations
timestamp	TEXT	ISO datetime string (UTC)
ph	REAL	pH reading
turbidity	REAL	Turbidity in NTU
conductivity	REAL	Conductivity in $\mu\text{S}/\text{cm}$
chlorine	REAL	Chlorine residual in mg/L
temperature	REAL	Temperature in Celsius
label	TEXT	unlabelled / unsafe / safe

Column	Type	Purpose
confidence	REAL	0.0 to 1.0 — label confidence weight

illness_reports

Column	Type	Purpose
id	INTEGER	Auto-incrementing primary key
station_id	INTEGER	Which station was reported
report_timestamp	TEXT	When the SMS was received
reporter_hash	TEXT	SHA-256 hash of phone number (first 16 chars)
confidence_weight	REAL	0.3 / 0.6 / 0.9 based on report count

9. Confidence weighting logic

Each illness report is assigned a confidence weight based on how many reports have been received from that station in the past 7 days. This handles the noisy label problem — a single report might be mistaken, but multiple independent reports from the same station are more reliable.

Reports in past 7 days	Confidence weight	Interpretation
0 (this is the first)	0.3	Single report — treat with caution
1 previous report	0.6	Two independent reports — likely genuine
2 or more previous	0.9	Multiple reports — high confidence

The labelling function also respects existing confidence — if a reading has already been labelled with a higher confidence weight, a lower-confidence new report will not overwrite it.

10. Exporting labelled data for ML training

Once enough labelled data has accumulated, export it to CSV for model training using the following Python snippet:

```
from labels import export_labelled_dataset
from database import get_db

n = export_labelled_dataset(get_db())
print(f'Exported {n} rows to labelled_dataset.csv')
```

The exported CSV contains: station_id, timestamp, ph, turbidity, conductivity, chlorine, temperature, label, confidence. This is the input to the XGBoost training pipeline.

Only rows with label != 'unlabelled' are exported. Unlabelled readings are retained in the database but excluded from training data.

11. Known limitations of this prototype

Limitation	Impact	Future fix
SQLite not suited to concurrent writes	Fine for prototype, will break under load	Migrate to PostgreSQL for production
No authentication on /reports and /stations	Anyone can read report data	Add API key middleware
Illness reports use incubation window only	Does not account for time of day or water usage patterns	Refine labelling window in v2
No deduplication of reporter phone numbers	Same person reporting twice counted as two reports	Track reporter hash per station per window
SMS only — no WhatsApp fallback	Misses smartphone users in some regions	Add WhatsApp Business API as second channel

12. Questions

If anything in this document is unclear, the full codebase with inline comments is available in the GitHub repository. Each function has a docstring explaining its purpose and expected inputs.

The code for this prototype was generated and tested — all tests passed including database initialisation, station ID parsing, and the retrospective labelling logic.